

Profilage Parcoursup

Créer Compte

Dossiers

Groupes

Formations

Filtres des dossiers

ANNÉE

Année de candidature

Sélectionnez...

CANDIDAT

Civilité

Sélectionnez...

Statut Boursier

Sélectionnez...

Profil

Sélectionnez...

GROUPE

Groupe

Sélectionnez...

DIPLÔME / BAC

Type de Bac

Sélectionnez...

SÉRIE DE BAC

Série

Sélectionnez...

NOTES

Note Lycée

Min >= Max

Note Fiche Avenir

Min >= Max

Note Globale

Année(s) sélectionnée(s) : 2026

Nombre de dossier(s) : 1342

↓ ↑

Code	Civilité	Code boursier	Note de lycée	Note de fiche avenir	Note de globale	Etablissement	Diplôme	Spécialité 1	Spécialité 2	Spécialité 3	Tout sélectionner
102374	M.	1	12.1	17	3.74	Lycée Claude Fauriel	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☐
102779	M.	0	13.46	20	4.26	Lycée général et technologique Henri Bergson	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie	☐
105442	M.	1	10.52	11	10.68	Lycée Général et Technologique Joséphine Baker	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie	☐
107816	M.	0	9.93	8	2.53	Lycée Andre Bouloche	Générale	Mathématiques Spécialité	Sciences de la vie et de la Terre Spécialité	Non définie	☐
108204	M.	1	11.74	14	3.41	Lycée Les Bruyères	Générale	Langues, littératures et cultures étrangères et régionales	Mathématiques Spécialité	Non définie	☒
109402	M.	0	13.73	17	4.04	Campus Sainte Marguerite	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie	☐
111534	M.	1	14.23	20	4.4	Lycée André Malraux	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☐
111564	M.	0	10.38	11	2.89	Lycée André Malraux	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☒
111598	M.	0	10.52	6	9.01	Lycée André Malraux	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☐
112956	Mme	0	13.68	14	13.79	Lycée St Louis Ste Clotilde	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☐
113211	M.	0	11.98	12.5	12.16	Lycée Jean Guehenno	STI2D	Physique-Chimie et Mathématiques	Ingénierie, innovation et développement durable	Non définie	☐

Affichage de la page 1 sur 54

Début Précédent 1 Suivant Fin

Carte

Valider

RAPPORT DE RÉALISATION

Année universitaire 2025-2026

Membre du groupe :		
HERMILLY	Joshua	G2
LAFOSSE	Lucas	G1
MILLEREUX BIENVAULT	William	F2
PRÉVOST	Donovan	G2

Prestataires :	
CHARRIER	Rodolphe
KHRAIMECHE	Salim

Table des matières :

I. Introduction.....	6
A. Contexte.....	6
B. Répartition.....	6
II. Interface Utilisateur.....	7
A. Page de connexion.....	7
B. Page d'inscription.....	7
C. Page dossiers.....	8
D. Les filtres.....	9
E. Le Tableau.....	10
F. Import - Export.....	10
G. La Carte.....	11
H. Asset Recherche □ Information.....	12
I. Mise en places des assets.....	12
J. Fonction groupe.....	13
K. Vue Utilisateur.....	15
III. Base de données.....	16
A. La conception initiale :.....	16
B. Une mauvaise interprétation des données :.....	17
1. Clarification des Données Élèves et Diplômes :.....	17
2. Recentrage sur le Parcours de l'Élève :.....	17
3. Gestion des Comptes Utilisateurs et des Groupes :.....	17
4. Gestion des Coordonnées.....	17
C. L'insertion en base de données :.....	19
1. L'arborescence du code d'insertion.....	19
2. L'importation de données.....	20
a. Choix Technologique.....	20
b. Lecture et Pré-tri des Données.....	20
c. Insertion par lots.....	21
d. Résumé.....	22
E. Exportation des données.....	22
1. Le choix de technologies :.....	22
2. Un DTO spécifique pour l'agrégation de données.....	22
3. La lecture et la compilation des données.....	22
IV. Les Filtres.....	23
A. Contexte et objectifs des filtres.....	23
1. Position du problème.....	23
2. Objectifs de conception.....	23
B. Vue d'ensemble de l'architecture des filtres.....	24
1. Chaîne de traitement.....	24
2. Réutilisation de l'architecture.....	24
C. Construction modulaire des filtres côté back-end.....	25
1. Service de configuration des filtres.....	25
2. Rôle de la vue Twig.....	25

3. Hydratation des options par le repository de filtres.....	25
4. Réutilisation pour les groupes.....	26
D. Traduction des filtres en requêtes SQL dans les repositories.....	26
1. Construction dynamique de la requête.....	26
2. Gestion des filtres simples.....	26
3. Gestion des notes par intervalles.....	26
4. Options de spécialité.....	26
5. Filtre de distance dans les requêtes.....	27
E. Gestion des filtres côté client en JavaScript.....	27
1. Centralisation dans tableau.js.....	27
2. Sérialisation des filtres et appel au back-end.....	27
3. Mise à jour du tableau et des indicateurs.....	27
4. Comportement au chargement initial.....	27
5. Dynamique des spécialités côté client.....	28
6. Autres scripts liés aux filtres.....	28
F. Cas particuliers, problèmes rencontrés et évolutions.....	28
1. Spécialités de bac : ajustement de la logique.....	28
2. Filtre de distance : comportement par défaut et UX.....	28
V. Tableau.....	29
A.l'asset.....	29
B.Génération des lignes.....	29
VI. Gestion des groupes : création et modification.....	31
A. Contexte et objectifs.....	31
1. Position du problème.....	31
2. Objectifs de conception.....	31
B. Vue d'ensemble de la chaîne de traitement.....	31
1. Scénario général.....	31
2. Rôle des contrôleurs.....	31
3. Rôle du service et des repositories.....	32
C. Architecture des fichiers liés aux groupes.....	33
1. Organisation back-end.....	33
2. Vues et scripts côté client.....	33
D. Détails de la création et de la modification.....	34
1. Création de groupe côté client.....	34
2. Modification et synchronisation des membres.....	34
VII. La carte, du back au front.....	35
A. Fichiers utilisée.....	35
B. Choix technique.....	35
1. Les API utilisées :.....	35
2. La carte choisi :.....	36
C. utilisation d'api.....	36
D. La création de la map.....	37
VIII. Apprentissage retenue.....	39
IX. Conclusion.....	39

Table des illustrations:

Numéro	Titre de la figure	Page
Figure 1.1 ▾	Graphique en anneau de la répartition du travail ▾	6 ▾
Figure 1.1 ▾	Graphique en barre horizontal de la répartition des tâches ▾	6 ▾
Figure 2.1 ▾	Page de connexion. ▾	7 ▾
Figure 2.2 ▾	Page d'inscription. ▾	8 ▾
Figure 2.3 ▾	Page des dossiers. ▾	8 ▾
Figure 2.4 ▾	Les filtres ▾	9 ▾
Figure 2.5 ▾	Le tableau ▾	10 ▾
Figure 2.6 ▾	Asset Import-Export ▾	10 ▾
Figure 2.7 ▾	Page Import-Export ▾	11 ▾
Figure 2.8 ▾	La Carte ▾	11 ▾
Figure 2.9 ▾	Panel Info ▾	12 ▾
Figure 2.10 ▾	Mise en place des assets ▾	13 ▾
Figure 2.11 ▾	Visualisation d'un groupe ▾	14 ▾
Figure 2.12 ▾	Modification du groupe ▾	14 ▾
Figure 2.13 ▾	Vue Utilisateur ▾	15 ▾
Figure 3.1 ▾	Première version de la base de données. ▾	16 ▾
Figure 3.2 ▾	Une base de données qui nécessite des changements rapides. ▾	18 ▾
Figure 3.3 ▾	La base de données suite aux changements. ▾	18 ▾
Figure 3.4 ▾	Arborescence insertion/exportation ▾	19 ▾
Figure 3.5 ▾	Capture d'écran du code de transformation du fichier en objet (t... ▾	20 ▾
Figure 3.6 ▾	Capture d'écran du code de tri des établissements. ▾	20 ▾

Figure 3.7 ▾	Capture d'écran de l'ajout par lots dans CandidatRepository. ▾	21 ▾
Figure 2.8 ▾	Schéma fonctionnel de l'importation de données via la feuille de... ▾	22 ▾
Figure 4.1 ▾	Schéma du chemin des données du filtre ▾	24 ▾
Figure 4.2 ▾	Schéma de l'architecture du filtre ▾	25 ▾
Figure 5.1 ▾	Une base de données qui nécessite des changements rapides. ▾	29 ▾
Figure 5.2 ▾	Capture d'écran pour disperser les données reçues de l'API du ... ▾	30 ▾
Figure 5.3 ▾	Capture d'écran expliquée du fetch pour le tableau ▾	30 ▾
Figure ▾	Schéma fonctionnel de l'appel de l'API et la réception de donnés... ▾	30 ▾
Figure 6.1 ▾	Schéma du chemin des données de la création et modification ... ▾	32 ▾
Figure 6.1 ▾	Schéma du chemin des données du filtre ▾	33 ▾
Figure 7.1 ▾	Capture d'écran de l'arborescence pour la carte. ▾	35 ▾
Figure 7.2 ▾	Capture d'écran de l'appel de notre API pour les données des é... ▾	36 ▾
Figure 7.3 ▾	Schéma fonctionnel de l'appel des API géocode. ▾	37 ▾
Figure 7.5 ▾	Données renvoyées par l'API en format JSON. ▾	37 ▾
Figure 7.6 ▾	Ajouter un marqueur sur la map. ▾	38 ▾

I. Introduction

A. Contexte

Ce travail fait partie du module S4.01 Développement d'une application ([S4.01 Développement d'une application](#)) de la 2ème année du BUT INFO. Les prestataires de ce projet sont M. Rodolphe CHARRIER et M. Kamil KHRAIMECHE.

Pour ce faire, notre groupe de 4, composé de Joshua HERMILLY, Lucas LAFOSSE, Donovan PRÉVOST et William MILLEREUX BIENVault.

Avant toute chose, voici la fonction et les rôles de chaque membre :

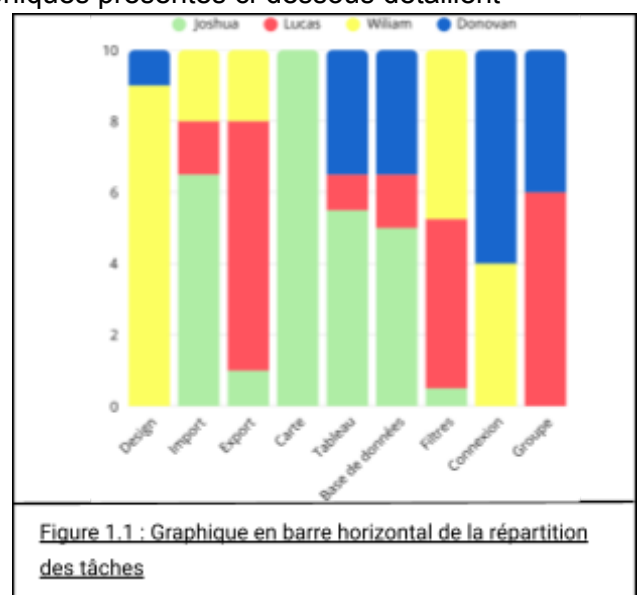
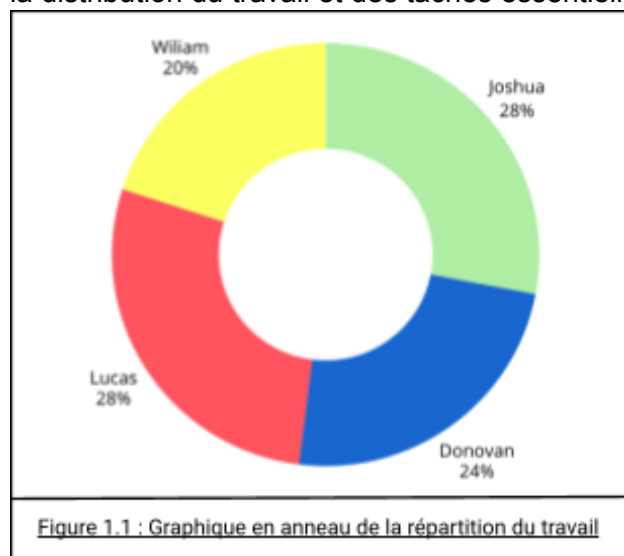
- Joshua : Organisateur & Concepteur - Chef de projet
- Lucas : Expert & Perfectionneur - Développeur full stack
- William : Propulseur & Coordinateur - Designer
- Donovan: Priseur & Soutien - Développeur

L'objectif principal de ce projet est le développement d'une application web complexe destinée au profilage des dossiers Parcoursup des candidats au BUT Informatique du Havre. Tout cela, dans le but de réduire le temps à mettre les notes de dossier des étudiants pour sélectionner les futurs 1ère année.

Pour ce rapport, nous allons donc présenter la réalisation de notre projet à-travers ce rapport de réalisation.

B. Répartition

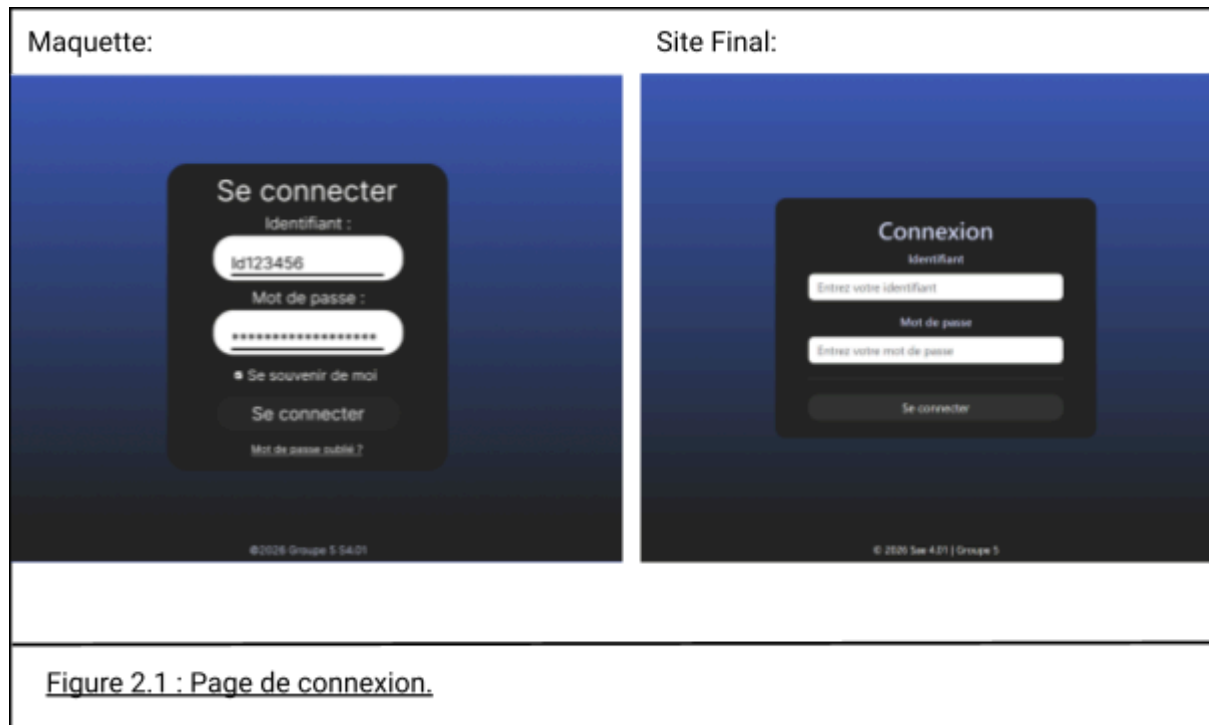
Pour la répartition du travail au sein du groupe, nous avons voulu garantir une charge de travail équitable entre tous les membres. Cette répartition a été basée sur la rapidité des tâches ainsi que sur le volume total de travail fourni. De plus, nous avons attribué les tâches en tenant compte des préférences et des compétences de chacun, pour les personnes les plus à l'aise ou désireuses de s'en charger. Les graphiques présentés ci-dessous détaillent la distribution du travail et des tâches essentielles.



II. Interface Utilisateur

A. Page de connexion

Nous avons débuté par les pages de login afin de vérifier si le style était fiable et cohérent. Suite à cela nous avons réalisé le style.css qui sera commun à toutes les pages qui va définir la charte graphique du site.

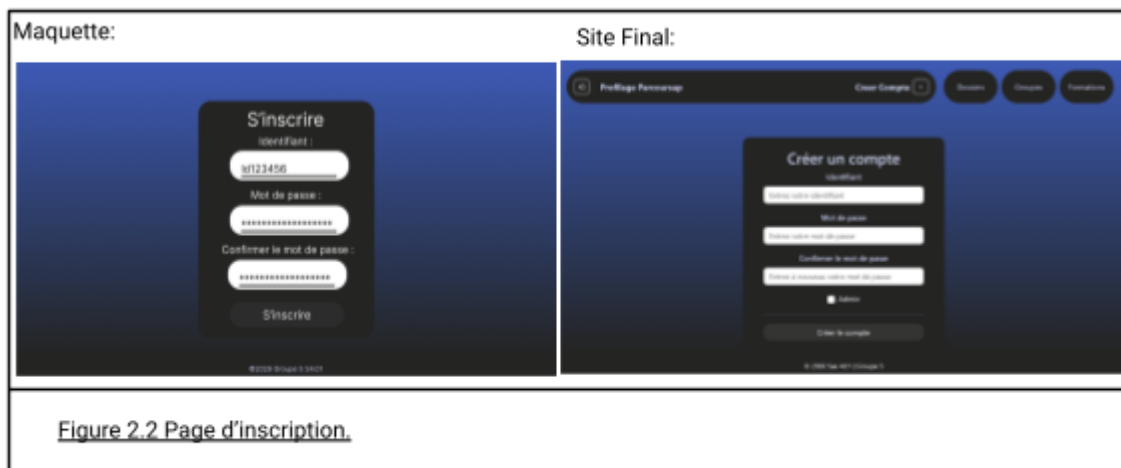


Celui-ci est donc plutôt proche de la maquette. Sur la maquette nous pouvions y voir la possibilité de changer de mots de passe qui a été abandonné en raison de la trop grande complexité pour le peu de temps que nous avons à notre disposition pour le réaliser. Et également un checkbox qui permettait de garder enregistrer les données mais qui se fait automatique, c'est-à-dire que actuellement si une personne s'est déjà connectée sur cette session elle sera directement amenée sur la page "Dossiers" qui est la page principale.

B. Page d'inscription

À la suite nous avons enchaîné avec 2 autres pages similaires : "creerCompte" et "password".

"Password" a été annulée pour les raisons présentées plus haut mais par contre creerCompte à été créer :



Sur celle-ci on retrouve la même structure que sur la maquette : identifiant / Mot de passe / Confirmation. Mais en plus de ça on retrouve la possibilité de définir si la personne sera admin. Ainsi, seuls les admins pourront créer des comptes pour les autres personnes. Nous avons également décidé de mettre le header car si l'administrateur souhaite revenir sur les autres pages il n'en avait pas la possibilité avec la maquette.

C. Page dossiers

Suite à ça, nous avons réalisé la page principale du site, la page des dossiers. Sur celle-ci nous avons commencé par réaliser le header et le footer. Et puisque nous voulions réaliser presque les mêmes pages, nous avons décidé de faire un fichier "base" que nous "étendons" dans chacune des pages. Dans ce fichier "base" on y retrouve les bases de la structure de toutes nos pages (head / header / footer / agencement des assets / background / titre de la page / import js et css).

Pour faire cette page dossiers avait à la base une structure avec sous header mais finalement celle-ci a été annulée pour laisser place à une structure de en ligne ou les éléments sont placés les uns après les autres pour faciliter la responsivité. Les seules bloqué en dehors sont le filtre et la carte ouverte afin de les afficher avec le js (à gauche et à droite du tableau).



D. Les filtres

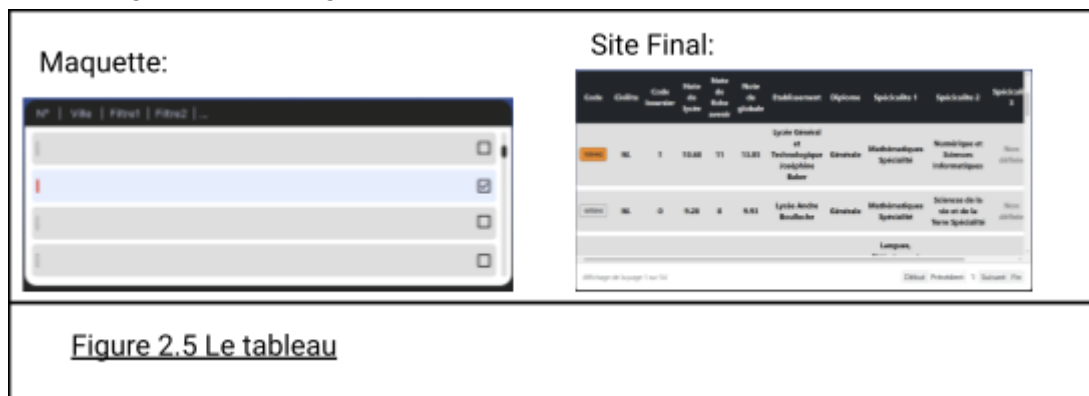
Tout d'abord, nous avons décidé de faire fonctionner notre site avec des assets que nous appellerons sur les différentes pages si on en a besoin, car encore une fois elles seront très similaires. Et dans un 2eme temps, quitte à faire en sorte de ne pas dédoubler notre code pour les assets, nous nous sommes dit que rendre les filtres modulaires serait plus pratique en termes de conception. Une fois fait pour la 1ere page, le filtre n'aura pas à être refait pour les autres pages : c'est géré facilement en "back". Mais ça a également un intérêt pour la maintenance : par exemple, si d'autres filtres doivent être ajoutés dans le futur ou si certains doivent être enlevés, ce sera beaucoup plus rapide et simple de les modifier.



Comme on peut l'observer sur le résultat, l'idée est la même mais la réalisation est différente, car comme on peut le voir sur la maquette de la page dossier, nous avons décidé de faire une page fixe afin de faciliter l'ergonomie et la compréhension du site. Ainsi, un premier point qui diffère est le fait que, pour mettre tous les filtres, nous avons dû faire un filtre scrollable. Et pour appliquer les filtres, nous nous sommes facilités la vie en réalisant le filtre comme un formulaire et non en adaptant le filtre dynamiquement avec du js. Ce qui explique l'apparition du bouton "Appliquer" en bas du filtre.

E. Le Tableau

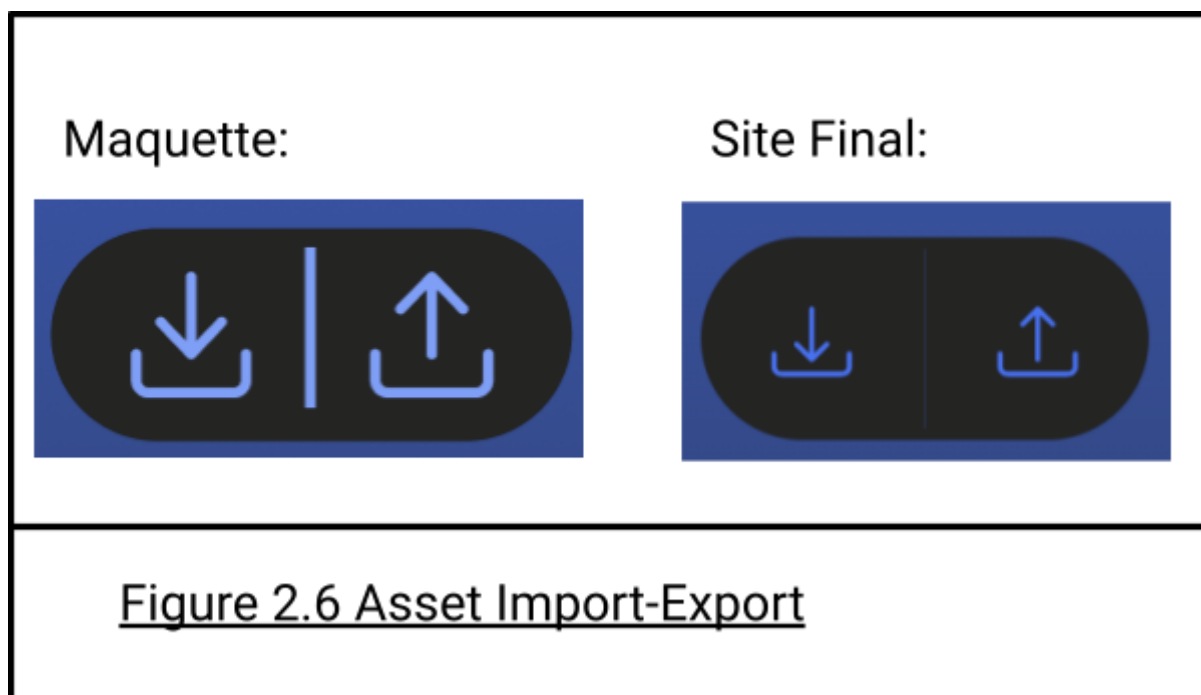
À la suite, nous avons réalisé le tableau afin de vérifier si les filtres fonctionnaient bien. Comme on peut le voir sur la maquette, celle-ci ne comportait pas de pagination, pas de slider horizontal, pas de pastilles de couleur des groupes sur le code et pas de surlignage de la ligne lors d'une sélection. Toutes ces modifications ont été apportées, toujours dues à un souci d'ergonomie et de gain de place.



Et comme avec le filtre, celui-ci est modulaire : il s'adapte en fonction des informations qu'on lui donne. Ainsi, on a pu le réutiliser pour les 3 pages sans faire de doublon de code.

F. Import - Export

Après celle-ci, ce sont les boutons et pages d'import et d'export qui ont été réalisés. Ils sont plutôt fidèles aux originaux. Mais les boutons finaux permettent d'amener à de nouvelles pages pour réaliser leur mission et non de le faire automatiquement en cliquant juste dessus.



Celles-ci permettent de définir l'année de la promotion qu'on souhaite importer ou exporter. Et pour l'import, on peut visualiser le fichier importé avant de valider.



Figure 2.7 Page Import-Export

G. La Carte

Après ceux-ci, nous avons commencé la carte, qui est également plus fine pour permettre de faciliter la lecture du tableau. Celle-ci fonctionne finalement avec un filtre de distance en km qui permet de filtrer les personnes qui sont plus ou moins loin de l'IUT. Et nous n'avons pas eu le temps de réaliser la possibilité de mettre la carte en grand.



Figure 2.8 La Carte

H. Asset Recherche ☐ Information

Nous avons également prévu de mettre une barre de recherche à l'origine, mais comme celle-ci a été considérée comme inutile, nous l'avons remplacée par un asset info qui permet de désigner les années et le nombre de dossiers pris en compte par le filtre actuel (si aucun filtre n'est sélectionné, il affiche toutes les années et le nombre de dossiers).

Maquette:



Site Final:

Année(s) sélectionnée(s) :	Nombre de dossier(s) :
2026	1343

Figure 2.9 Panel Info

I. Mise en places des assets

À la suite de ça, tous les assets de la page "Dossiers" ont été réalisés, ce qui fait qu'il a seulement été nécessaire de placer les assets Twig dans le bon ordre et de leur fournir les bonnes informations pour créer la majorité de la page :



Figure 2.10 Mise en place des assets

Comme on peut le voir sur les pages finales de groupes et de formations, l'idée d'origine des maquettes n'est pas du tout la même et, en l'occurrence, nous ne comptons pas adapter les filtres pour le groupe ni mettre l'import et l'export, car tout cela serait inutile.

J. Fonction groupe

Dans les fonctions que nous avons rajoutées, on retrouve la possibilité de filtrer en fonction des données du tableau des groupes. Et détail important : la visualisation qui affiche les dossiers avec les filtres utilisés pour créer ce groupe,

Filtres des dossiers

ANNÉE
Année de candidature
Sélectionnez...

CANDIDAT
Civilité
Mme
Statut boursier
Sélectionnez...
Profil
Sélectionnez...

Année(s) sélectionnée(s) : 2026 Nombre de dossier(s) : 2

Code	Civilité	Code boursier	Note de lycée	Note de fiche avenir	Note de globale	Etablissement	Diplôme	Spécialité 1	Spécialité 2	Spécialité 3
788796	Mme	0	18.32	20	19.69	Lycée Guy De Maupassant	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie
778875	Mme	0	17.62	20	19.57	Lycée Guy De Maupassant	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie

Affichage de la page 1 sur 1

Début Précédent 1 Suivant Fin

Figure 2.11 Visualisation d'un groupe

et l'édition des groupes, qui ouvre la page "Dossiers" et permet de changer la couleur, le nom, la note, ainsi que d'ajouter ou de supprimer des personnes en les sélectionnant ou non.

Mode édition de groupe : Groupe Grp des Madame

Grp des Madame 20 Quitter Enregistrer

Code	Civilité	Code boursier	Note de lycée	Note de fiche avenir	Note de globale	Etablissement	Diplôme	Spécialité 1	Spécialité 2	Spécialité 3	Tout sélectionner
141873	Mme	0	17.22	20	15.83	Lycée Ste Therese	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie	☐
820883	Mme	0	17.56	20	16.34	Lycée Peter Pan	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie	☐

Affichage de la page 1 sur 1

Début Précédent 1 Suivant Fin

Figure 2.12 Modification du groupe

K. Vue Utilisateur

Et enfin voici le point de vue utilisateur :



Figure 2.13 Vue Utilisateur

Ce qui fait que les cases à cocher et les boutons situés sous "Appliquer" dans le filtre ont été retirés, car ils permettaient la gestion des groupes. Également, les boutons import et export n'ont pas été remis, car on considère que les utilisateurs peuvent seulement visualiser les groupes et non les créer. Enfin, la création de compte a également été retirée pour les non-admins.

III. Base de données

Cette partie présente la réalisation de notre base de données. Nous y détaillerons la phase de conception initiale jusqu'à la mise en œuvre finale, en passant par la résolution des problèmes rencontrés.

A. La conception initiale :

Le modèle conceptuel de la base de données était autour de 4 axes : **Candidat**, **Année Actuelle** (regroupant Établissement, Formation, Spécialité), **Diplôme**, et **Filtre/Groupe** pour la notation.

Deux règles de gestion sont établies :

- **Note de Dossier** dépend de l'appartenance à un **Groupe** défini par un **Filtre**.
- **Statut Académique** 2024/2025 (poursuite d'études, redoublement, etc.) est déduit des informations renseignées (Filière, Spécialité, Établissement d'origine 2024/2025).

Pour visualiser cette base voici notre diagramme initiale :

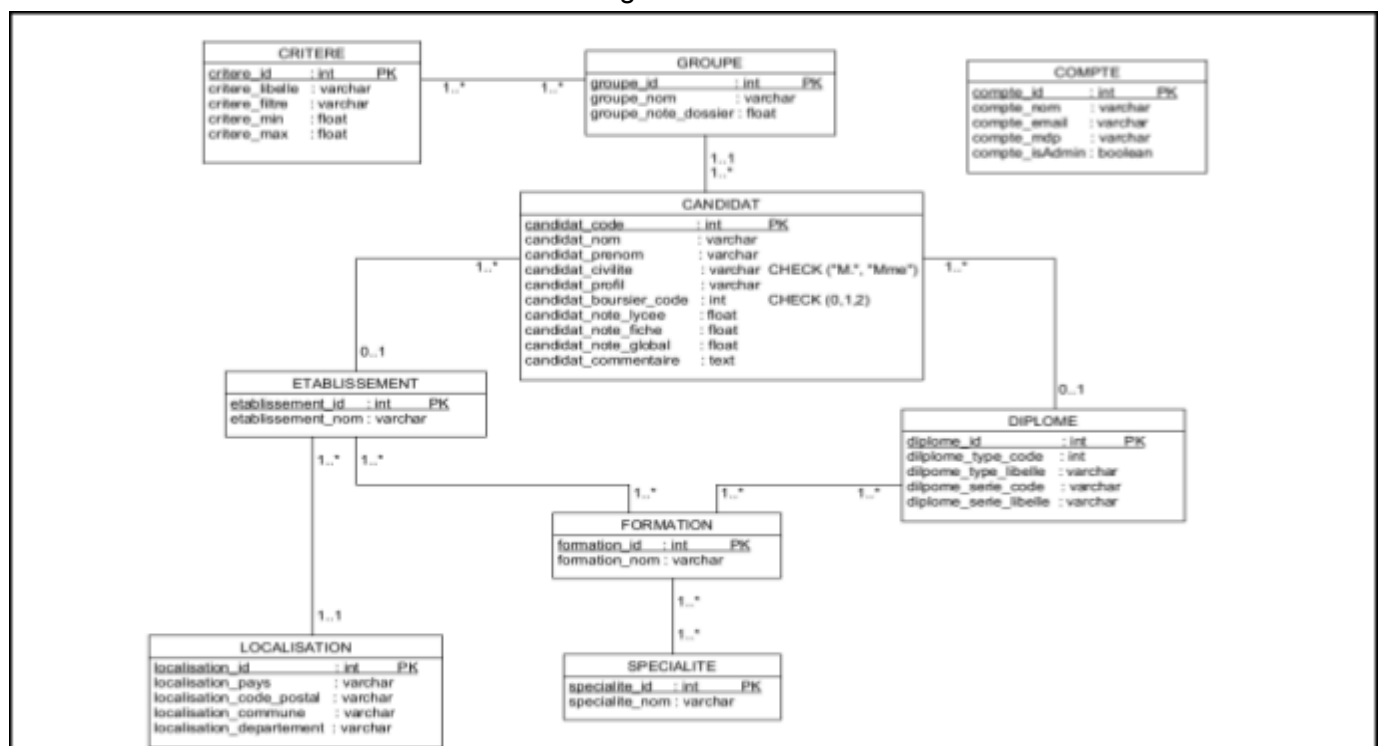


Figure 3.1 : Première version de la base de données.

B. Une mauvaise interprétation des données :

Évolution et Corrections de la Conception de la Base de Données

La conception initiale de la base de données présentait plusieurs incohérences, particulièrement sur la gestion des redoublements et les liens entre un diplôme, une formation, et un établissement.

1. Clarification des Données Élèves et Diplômes :

Suite à l'intervention de M. KRAIMECHE, nous avons rectifié une mauvaise interprétation des données issues du fichier Excel. Il s'est avéré que les colonnes intitulées « Spécialité / Mention - Libellé 2024/2025 » et « Spécialité - Libellé » ne représentaient pas un possible redoublement, mais plutôt les options choisies en Première et Terminale.

Cette clarification a conduit à 2 ajustements :

- Il n'existe qu'1 liaison entre un élève et son diplôme.
- Le diplôme n'est plus directement rattaché à une formation, mais à une spécialité qui à divers enseignements et options (Première/Terminale).

2. Recentrage sur le Parcours de l'Élève :

Nous avons également redéfini le rattachement de l'établissement. La liaison n'est plus faite entre la formation et l'établissement, mais **directement avec l'élève**. Cette modification met l'accent sur la situation individuelle de l'élève (« où tu es toi ») et son historique de parcours (« où tu as étudié et qu'as-tu fait »), plutôt que sur l'établissement dispensant la formation.

3. Gestion des Comptes Utilisateurs et des Groupes :

Enfin, le compte utilisateur a été reconsidéré. Il ne s'agit plus d'un simple « rôle », mais d'une entité à part entière connectée à la base de données. Un compte utilisateur peut désormais être le créateur de zéro ou plusieurs groupes.

Nous avons également ajouté un attribut couleur : varchar à l'entité Groupe. Cet attribut est prévu pour permettre l'affichage différencié des groupes au niveau de l'interface utilisateur.

Ces premières corrections et ajustements logiques nécessitent la création d'un nouveau diagramme Entité-Association révisé.

4. Gestion des Coordonnées

Suite au développement de la fonctionnalité de cartographie, nous avons identifié une limitation dans la modélisation de la localisation des établissements. Initialement, la localisation était gérée par une entité séparée (**Localisation**) liée à l'entité **Établissement**.

Cette structure s'est avérée inefficace pour représenter la relation "quel établissement se trouve à quelle localisation". Pour simplifier la modélisation et optimiser les requêtes, nous avons donc fusionné les informations :

- L'entité **Localisation** a été supprimée.
- Les attributs de localisation (**Pays, Code Postal, commune, Département, Latitude, Longitude, Distance**) ont été directement intégrés à l'entité **Établissement**.

Cette modification permet une récupération des coordonnées plus directe, essentielle pour l'affichage précis sur la carte de l'interface utilisateur.

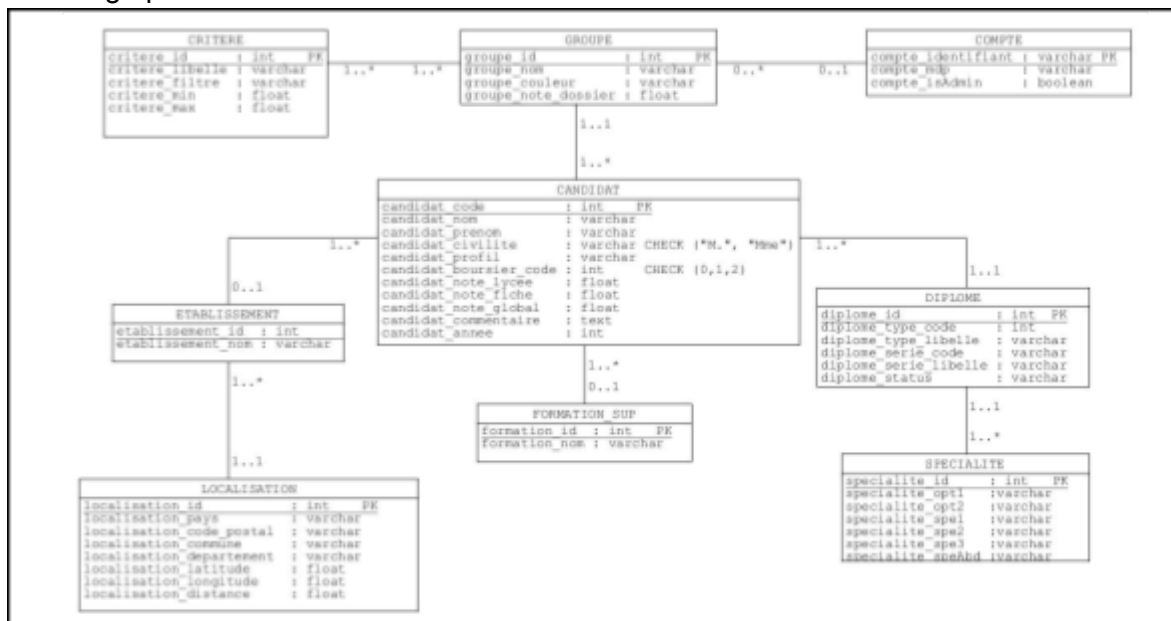


Figure 3.2 : Une base de données qui nécessite des changements rapide.

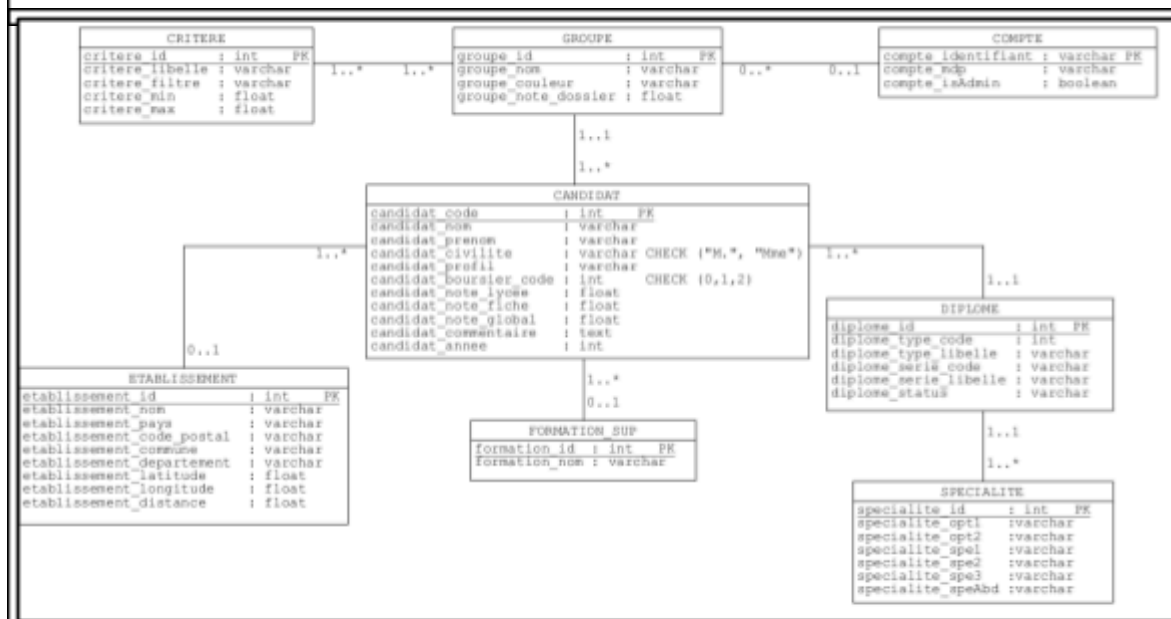


Figure 3.3 : La base de données suite au changements.

C. L'insertion en base de données :

Pour cette partie, nous allons expliquer comment nous avons fait pour insérer les données fournies par le tableur en base de données.

1. L'arborescence du code d'insertion

Voici l'arborescence de notre projet relative à la gestion de l'importation et de l'exportation. Pour préciser : les repositories sont appelés par le service, qui est lui-même appelé par le controller.

Un Repository gère l'accès aux données, représentant une entité de la base de données ou un DTO spécifique pour l'export.

Un Service contient la logique métier et utilise les repositories pour effectuer les requêtes.

Le Controller est le point d'entrée de la méthode, responsable de la gestion des connexions, des requêtes entrantes et des éventuelles redirections.

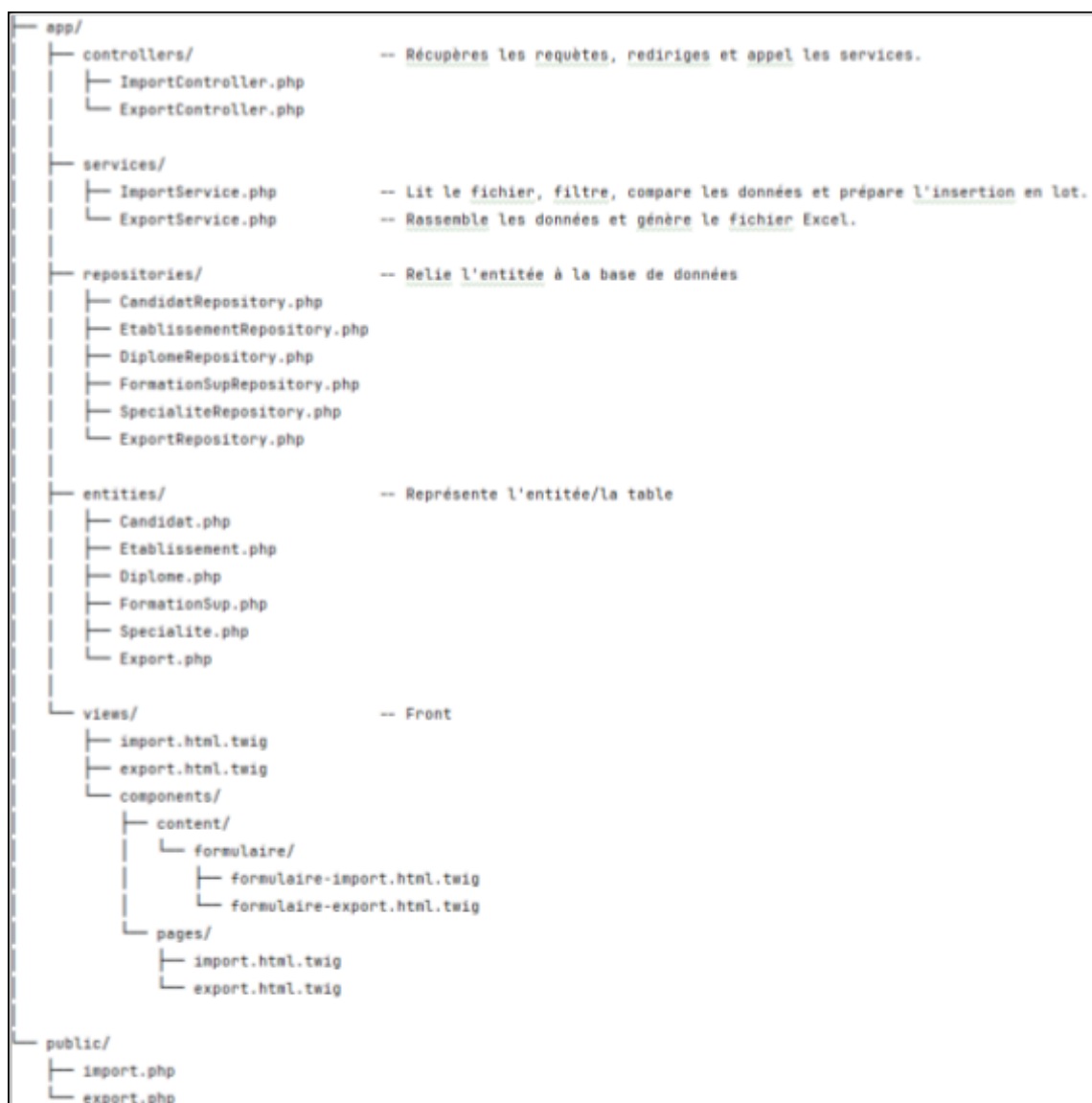


Figure 3.4 : Arborescence insertion/exportation

2. L'importation de données

L'importation de données sera réalisée via le chargement d'un fichier tableur, lequel est associé à une année spécifique.

a. Choix Technologique

Plusieurs options pour manipuler des fichiers (tableur) étaient envisageables, telles que [SheetJs](#) ou [PhpSpreadsheet](#). Notre choix s'est porté sur **PhpSpreadsheet**. Cette décision est motivée par la simplification et la cohérence qu'offre cette librairie, notamment dans l'interaction avec nos *repositories* directement en php sur le serveur et la centralisation de la logique métier, jugée plus directe.

Le service de chargement du fichier est implémenté dans [ImportService.php](#).

```

1 usage  & HERMILLY Joshua +2 *
public function importFile($file, $annee)
{
    $sheet = $file->getActiveSheet();           // récupérer la feuille associé au tableur
    $data = $sheet->toArray(null, true, true);    // récupérer les données de la feuille en un tableau
    array_shift( &array: $data);                 // supprime les entêtes

```

Figure 3.5 : Capture d'écran du code de transfoation du fichier en objet (tableau).

b. Lecture et Pré-tri des Données

Pour minimiser les échanges avec la base de données, nous préchargeons les données existantes. Ce pré-tri permet de comparer les entrées du tableur avec la base pour limiter les doublons. Les nouvelles entrées sont ajoutées à une liste d'entités à créer en base.

```

/*-----*/
/* Create */
/*-----*/
1 usage  & HERMILLY Joshua +1 *
private function createEtb(array $ligne)
{
    $etablissement = new Etablissement
    (
        etablissement_id: 0,
        $this->getCol($ligne, key: 'etab_nom' ),
        $this->getCol($ligne, key: 'pays' ),
        etablissement_code_postal: $this->getCol($ligne, key: 'commune_cp') == '99999' ? null : $this->getCol($ligne, key: 'commune_cp'),
        $this->getCol($ligne, key: 'commune_libelle'),
        $this->getCol($ligne, key: 'departement' ),
        etablissement_latitude: null,
        etablissement_longitude: null,
        etablissement_distance: null,
        nbr: null
    );

    foreach ($this->etablissements as $etab)
    {
        if ( $etablissement->getEtablissementNom() == $etab->getEtablissementNom() ) $G
        $etablissement->getEtablissementPays() == $etab->getEtablissementPays() ) $G
        $etablissement->getEtablissementCommune() == $etab->getEtablissementCommune() ) $G
        $etablissement->getEtablissementCodePostal() == $etab->getEtablissementCodePostal() ) $G
        $etablissement->getEtablissementDepartement() == $etab->getEtablissementDepartement() )
        {
            return $etab;
        }
    }

    $this->nouveauxEtablissements[] = $etablissement;
    $this->etablissements[] = $etablissement;
    $this->serviceRechercheLocalisation->addEtablissement($etablissement);
    return $etablissement;
}

```

Figure 3.6 : Capture d'écran du code de trie des établissement.

c. Insertion par lots

Pour prévenir les erreurs de *timeout* PHP (requêtes dépassant 30 secondes), les insertions sont faites par lots. Nous utilisons la méthode `creates` de nos *repositories* pour effectuer ces insertions groupées.

Nous avons voulu une insertion par lots pour gagner en performances lors des ajouts et limiter les appels à la base. Le document décrit le processus d'importation de données à partir de fichiers tableurs.

```
1 usage  👤 Apscawolf +3
public function creates(array $candidats)
{
    if (empty($candidats)) { return; }

    $taillePartie = 500;
    $total        = count($candidats);

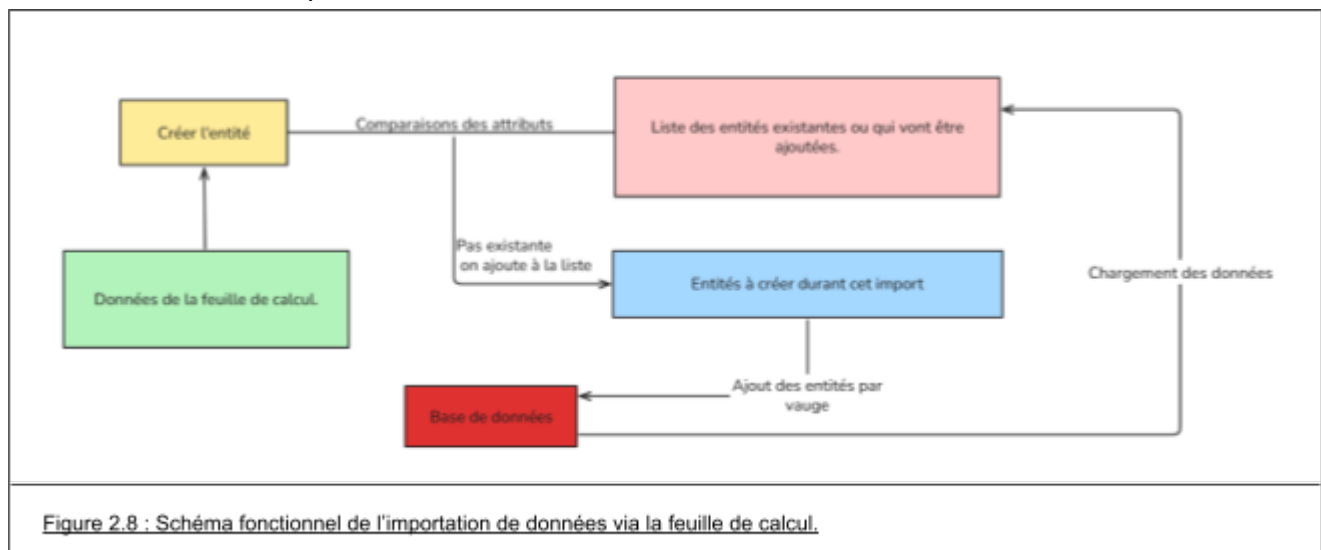
    for ($cptCdt = 0; $cptCdt < $total; $cptCdt += $taillePartie)
    {
        $partie = array_slice($candidats, $cptCdt, $taillePartie);
        $this->insererPartie($partie);
    }
}

1 usage  👤 HERMILLY Joshua +3
private function insererPartie(array $candidats): void
{
```

Figure 3.7 : Capture d'écran de l'ajout par lots dans CandidatRepository.

d. Résumé

1. Choix Technologique : PhpSpreadsheet.
2. Lecture et Pré-tri.
3. Insertion par lots.



E. Exportation des données

L'exportation de données permet de générer un fichier csv regroupant l'intégralité des informations des candidats pour une année de promotion spécifique.

1. Le choix de technologies :

Tout comme pour l'importation, nous utilisons [PhpSpreadsheet](#) pour garder une cohérence technique avec l'importation, la génération et le téléchargement de ce fichier se font dans **ExportService.php**.

2. Un DTO spécifique pour l'agrégation de données

Pour faciliter le transit et le formatage des données entre la base de données et la génération du fichier Excel , nous avons conçu une entité spécifique : **Export.php**.

Contrairement aux entités classiques du projet qui modélisent une seule table (comme Candidat ou Groupe), l'entité **Export** agit comme un DTO. Son rôle exclusif est de réceptionner et d'assembler des données provenant de multiples tables.

3. La lecture et la compilation des données

Afin d'optimiser les performances de l'exportation, nous avons choisi de limiter les interactions avec la base de données. Au lieu de faire plusieurs requêtes pour chaque candidat (pour récupérer son lycée, ses diplômes, etc...), l'**ExportRepository** exécute une seule requête SQL globale avec plusieurs jointures (**LEFT JOIN**) et des agrégations (**string_agg** pour regrouper les spécialités, par exemple). Les données récupérées sont passées au service, qui utilise *PhpSpreadsheet* pour écrire toutes les lignes dans un seul fichier Excel, avant d'initier son téléchargement sur le navigateur de l'utilisateur.

IV. Les Filtres

A. Contexte et objectifs des filtres

1. Position du problème

Au départ, pendant la phase de conception, nous avons surtout décrit les écrans et les interactions côté utilisateur. La mise en œuvre des filtres côté back-end a été peu détaillée.

Très vite, pendant la réalisation, nous nous sommes rendu compte que la partie filtrage serait en réalité centrale dans l'application : la plupart des écrans reposent sur des listes filtrables (dossiers et groupes), avec des composants réutilisables (assets Twig et JavaScript).

2. Objectifs de conception

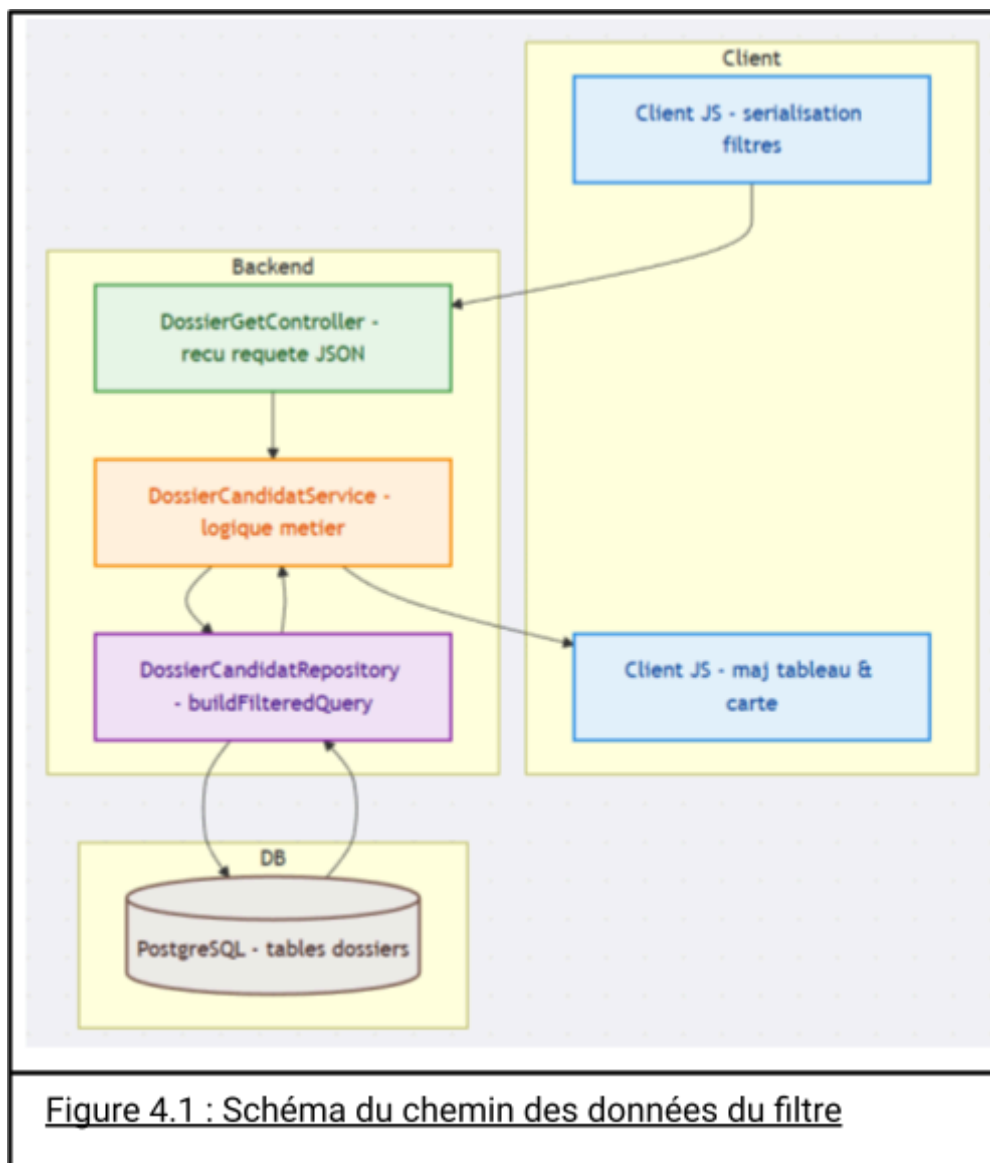
Notre objectif : construire un système de filtres modulaire, configurable et le moins "en dur" possible. Nous voulions pouvoir ajouter, modifier ou supprimer un filtre sans réécrire tout le code, et surtout sans dupliquer la logique entre le back-end et le JavaScript.

Nous avons également choisi de respecter l'architecture vue en cours (paternelle contrôleur – service – repository) afin de garder une séparation claire des responsabilités.

B. Vue d'ensemble de l'architecture des filtres

1. Chaîne de traitement

Le JavaScript sérialise les filtres saisis, les envoie en JSON à un contrôleur PHP via fetch. Le contrôleur appelle un service qui délègue la requête SQL au repository. Le repository traduit les filtres en conditions SQL, exécute la requête, et renvoie les résultats. Le service formate les résultats en entités, le contrôleur renvoie le tout en



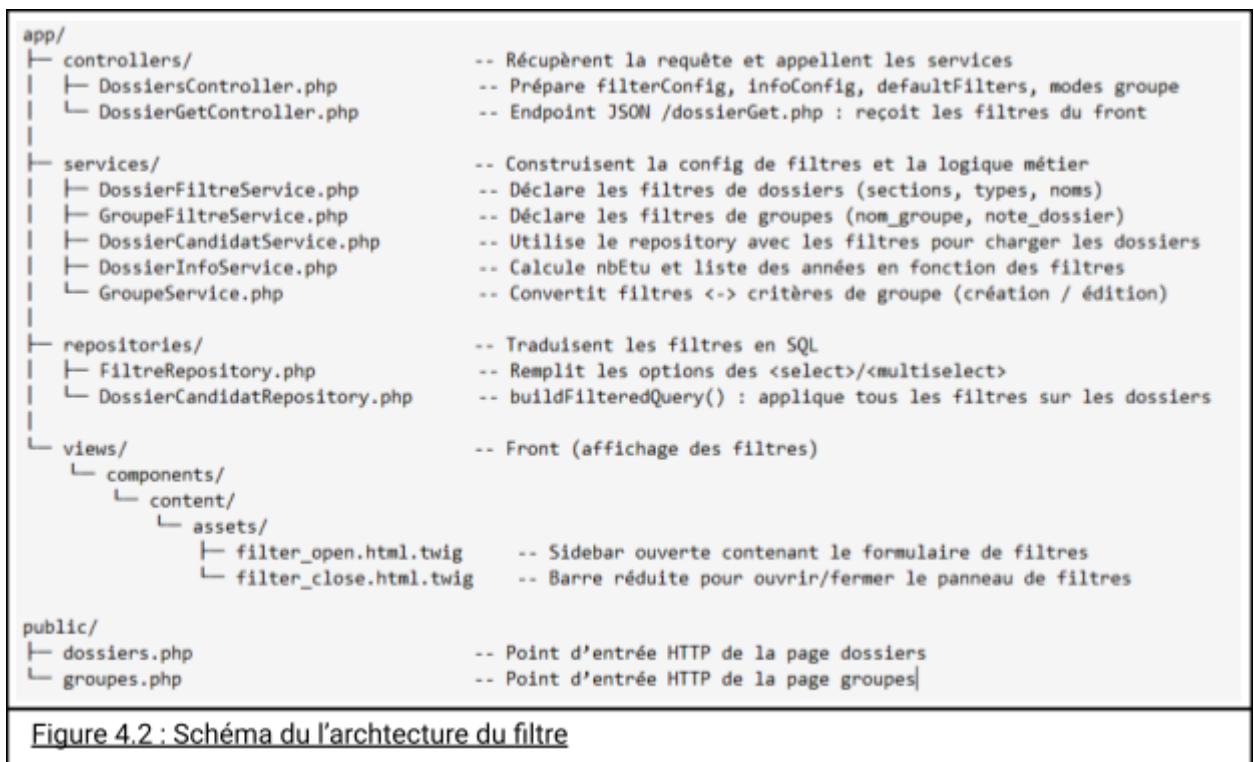
JSON, et le JavaScript met à jour l'affichage (tableau/carte).

2. Réutilisation de l'architecture

Cette structure se retrouve principalement dans les fichiers *DossierGetController.php*, *DossierCandidatService.php* et *DossierCandidatRepository.php* pour les dossiers.

Elle est également utilisée pour les groupes et les formations, avec d'autres services et repositories, mais en conservant les mêmes principes d'architecture.

Voici notre architecture :



C. Construction modulaire des filtres côté back-end

1. Service de configuration des filtres

Pour éviter de coder les filtres « en dur », le service *DossierFiltreService.php* a été créé.

Ce service ne fait pas de requêtes SQL ; il décrit la structure des filtres via un tableau associatif PHP. Chaque section du panneau (ex. « Candidat », « Bac », etc.) est une entrée du tableau. À l'intérieur, on définit les champs de filtre avec leur clé, type (select, multiselect, numérique, etc.) et options par défaut si besoin.

2. Rôle de la vue Twig

L'idée est de décrire "ce qu'est" un filtre plutôt que "comment l'afficher". La vue Twig lit cette configuration et génère dynamiquement les champs de formulaire.

Cela permet d'ajouter un nouveau filtre en ne modifiant que le service, sans dupliquer de balises HTML ni de JavaScript structurel.

3. Hydratation des options par le repository de filtres

Le *FiltreRepository.php* alimente automatiquement les listes déroulantes et cases à cocher en fournissant des requêtes SQL génériques pour récupérer les valeurs distinctes (promotions, types de bacs, groupes, spécialités, etc.). Le service de filtres

fusionne ces résultats dans la configuration, créant une source de vérité unique. Ainsi, toute évolution de la base de données met à jour les filtres automatiquement, sans modification côté *front* (vue ou JavaScript).

4. Réutilisation pour les groupes

Le même principe de configuration modulaire est réutilisé pour les filtres des groupes via `GroupeFiltreService.php`, qui s'appuie à la fois sur `FiltreRepository` et sur les repositories de groupes et de critères.

D. Traduction des filtres en requêtes SQL dans les repositories

1. Construction dynamique de la requête

La partie la plus technique se situe dans les repositories, en particulier `DossierCandidatRepository.php`.

La méthode centrale construit une requête SQL à partir d'un tableau de filtres. Elle concatène dynamiquement les clauses ``WHERE`` en fonction des clés présentes dans ce tableau.

2. Gestion des filtres simples

Pour les filtres simples (civilité, statut boursier, type ou série de bac, profil), le repository ajoute des conditions directes :

- égalité sur une colonne, ou
- recherche partielle avec ``ILIKE`` pour le profil.

Cette logique reste classique : si le filtre est présent et non vide, la méthode ajoute une condition SQL et un paramètre associé.

3. Gestion des notes par intervalles

Initialement, la recherche de candidats par spécialité utilisait une logique simple "OU" (au moins une spécialité sélectionnée dans *spe1*, *spe2* ou *spe3*), produisant des résultats trop larges.

La nouvelle approche est plus stricte : une logique "ET sans extra". Le candidat doit posséder exactement les spécialités demandées, ni plus, ni moins. La requête vérifie que chaque spécialité choisie est présente, et restreint les colonnes (*spe1*, *spe2*, *spe3*) uniquement à l'ensemble autorisé (plus *NULL*), garantissant ainsi une correspondance exacte.

4. Options de spécialité

Pour les options de spécialité (``specialite_opt1`` et ``specialite_opt2``), nous avons adopté une logique plus permissive.

Le repository cherche si au moins une des options choisies est présente dans les deux colonnes, en combinant des conditions avec "OU".

Ce comportement correspond à une recherche des candidats ayant au moins une des options cochées.

5. Filtre de distance dans les requêtes

Le filtre des distances est géré par la carte, avec un slider qui agit directement sur les filtres *distance_min* et *distance_max* du filtre.

E. Gestion des filtres côté client en JavaScript

1. Centralisation dans `tableau.js`

Côté client, la logique des tableaux est centralisée dans `tableau.js`.

Ce script gère les filtres, l'appel back-end et la mise à jour du tableau. Il récupère l'état du filtrage via les champs du formulaire, sans connaître les détails HTML.

2. Sérialisation des filtres et appel au back-end

La sérialisation convertit les champs de formulaire (valeurs simples et listes multiples) en un objet JSON pour le `FiltreRepository.php`.

Le clic sur "Appliquer" empêche le rechargement, récupère les filtres et lance `getData`.

Cette fonction envoie une requête fetch (ex: vers `dossierGet.php`) avec un corps JSON contenant la page et les filtres.

3. Mise à jour du tableau et des indicateurs

À la réception de la réponse JSON, `tableau.js` met à jour le tableau HTML :

- il reconstruit les en-têtes à partir de la liste renvoyée par le back-end,
- il remplit les lignes avec les données sérialisées par l'entité associé à la page
- il met à jour la pagination ainsi que les informations de synthèse (nombre total de candidats, années disponibles).

4. Comportement au chargement initial

Au chargement initial de la page des dossiers, nous avons pris une décision importante :

- le script enlève toute valeur de distance éventuellement restée dans `sessionStorage`,
- puis il charge la première page sans filtre de distance.

Cela garantit que l'utilisateur voit tous les dossiers au premier affichage, sans restriction cachée liée à une navigation précédente.

5. Dynamique des spécialités côté client

Le script *filter-bac.js* gère la dynamique des spécialités côté client. Il ajuste l'affichage et l'activation des cases à cocher selon le type et la série de bac sélectionnés, garantissant des combinaisons de filtres cohérentes. Ce script prépare les filtres pour le back-end sans modifier la logique métier.

6. Autres scripts liés aux filtres

Enfin, nous avons des scripts plus simples comme *filter-buttons.js*, qui s'occupent d'ouvrir ou de fermer le panneau de filtres, et *groupe.js*, qui réutilise les mêmes mécanismes de sélection de candidats pour créer et modifier des groupes en s'appuyant sur les filtres déjà appliqués.

F. Cas particuliers, problèmes rencontrés et évolutions

1. Spécialités de bac : ajustement de la logique

La partie filtrage a nécessité une évolution conceptuelle due à plusieurs problèmes. Initialement, pour les spécialités de bac, les requêtes SQL utilisaient une logique de type "OU", acceptant des candidats ayant l'une ou l'autre spécialité, y compris des non souhaitées. Après des tests avec des données réelles, ce comportement ne correspondant pas aux attentes, nous avons revu la condition SQL pour adopter une logique de type "ET".

2. Filtre de distance : comportement par défaut et UX

Le second cas est celui du filtre de distance. Initialement, la carte appliquait un filtre de distance dès le chargement, affectant les dossiers et pouvant exclure des candidats ou groupes involontairement. Un autre problème était la double validation requise pour appliquer le filtre à la fois sur la carte et dans le tableau.

Pour corriger cela, nous avons :

- Clairement séparé les responsabilités de la carte et du tableau.
- Permis à la carte de charger tous les établissements avec une valeur spéciale de distance.
- Fait en sorte que le tableau ne lise la distance qu'après validation explicite du curseur par l'utilisateur.
- Adapté le contrôleur de carte pour que les cas spéciaux (distance à -1 ou égale à la distance maximale) signifient "aucun filtrage par distance".

V. Tableau

A.l'asset

Tout d'abord, nous avons réalisé la structure du tableau grâce aux Twig, qui a permis de créer le contour du tableau ainsi que son initialisation. Nous avons également appliqué le style et les classes Bootstrap au tableau et sa bordure. Notre asset tableau est réparti en deux parties : la première est le tableau lui-même avec son en-tête fixe, ce qui permet de toujours le visualiser, et la deuxième partie, en bas, est la pagination, qui permet de limiter le nombre de candidats affichés à 25 à la fois. Cette pagination offre la possibilité de changer de page et d'afficher le nombre de pages disponibles.

Code	Civilite	Code boursier	Note de lycée	Note de fiche avenir	Note de globale	Etablissement	Diplome	Spécialité 1	Spécialité 2	Spécialité 3	Tout sélectionner
1267952	M.	0	8.5	NaN	NaN	lycée l'aouina	Générale	Mathématiques Spécialité	Numérique et Sciences Informatiques	Non définie	☐
1267282	Mme	0	NaN	NaN	NaN	ECE Paris	SCI	Non définie	Non définie	Non définie	☐
1267494	M.	0	14.19	NaN	NaN	Lycée François 1er	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☐
1267996	M.	0	NaN	NaN	NaN	Non définie	GEN	Non définie	Non définie	Non définie	☐
1269049	M.	0	NaN	NaN	NaN	Lycée Jules Sagna	SCI	Non définie	Non définie	Non définie	☐
1269136	M.	0	10.25	NaN	NaN	Lycée P'tit Michou	Générale	Mathématiques Spécialité	Physique-Chimie Spécialité	Non définie	☐

Affichage de la page 48 sur 54

Début Précédent 48 Suivant Fin

Figure 5.1 : Une base de données qui nécessite des changements rapide.

B.Génération des lignes

Le fichier **Tableau.js** est le fichier qui gère la création des lignes du tableau. La génération est modulaire donc chaque ligne est créée dynamiquement, quel que soit le contenu des headers.

Les données sont récupérées via un appel à notre api qui nous renvoie des données en json.

- Le header du tableau ('headers' => getHeader ())
- Les données du tableau avec ces colonnes ('donnees' => getDonnees())
- C'est un admin ('isAdmin' => isAdmin)

Comme mentionné, le design est modulaire. Le même tableau est utilisé pour afficher les formations, les groupes et les dossiers.

La structure du tableau est générée dynamiquement : chaque 'colonne' d'en-tête est créée comme un élément `<th>` et les données sont présentées dans des éléments `<tr>`.

```
if ( donnees['erreur'] ) { afficherErreur( donnees['erreur'] ); return; }

dvErreur.style.display = "none";
tableau.style.display = "";
creerHeader ( donnees['headers'], donnees['isAdmin'] );
creerTableau( donnees['headers'], donnees['data'], donnees['isAdmin'] );
creerBtnPage( donnees['maxPage'], donnees['actPage'] );
```

Figure 5.2: capture d'écran pour disperser les données reçu du l'api du tableau.

Pour appeler l'api nous avons un fetch lui aussi modulaire qui nous permet de ne pas dupliquer le code et ainsi garder la même persistance de données.

```
const response :Response = await fetch(Lien des api (formation, dossier, groupe)
init: {
  method : 'POST', Verbe HTTP de la méthode
  headers:
  {
    'Token' : 'SAE-4.01_WEB_TOKEN', Notre Token de vérification
    'Content-Type': 'application/json'
  },
  body: JSON.stringify( value: { page:indexPage, ...filters }) Paramètres du body
});
```

Figure 5.3: capture d'écran expliqué du fetch pour le tableau

La gestion des données s'effectue en deux phases :

Appel API et Traitement Back-end : Un appel à l'API du serveur back-end. Celui-ci renvoie les données correspondant à la requête, après les avoir filtrées.

Génération Dynamique du Tableau : Nous procédons ensuite à la création dynamique de notre tableau à partir des données reçues.

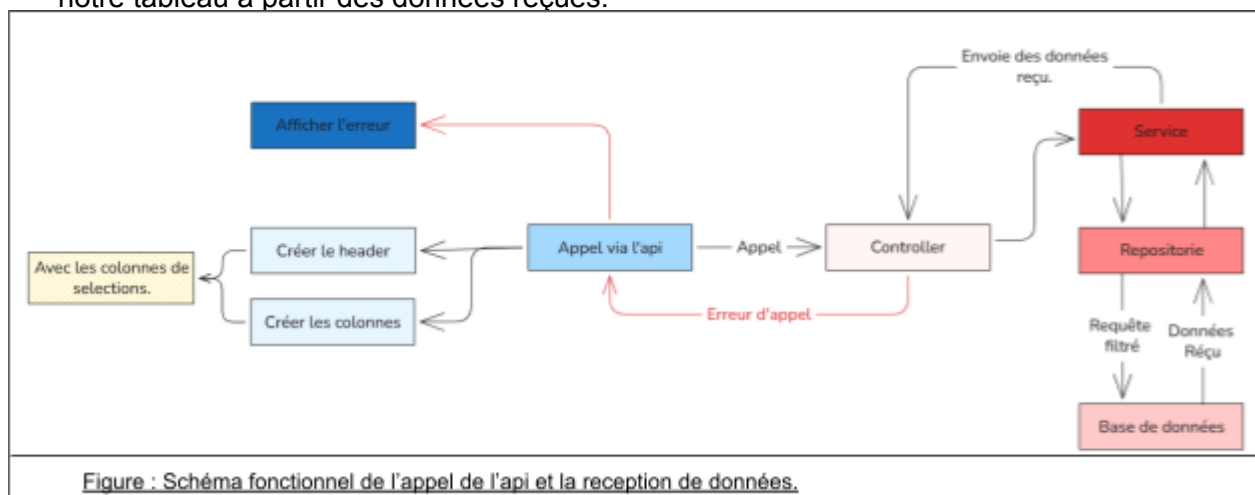


Figure : Schéma fonctionnel de l'appel de l'api et la reception de données.

VI. Gestion des groupes : création et modification

A. Contexte et objectifs

1. Position du problème

Initialement, les groupes devaient être de simples étiquettes appliquées à des dossiers, gérées par l'utilisateur via une interface simple.

Cependant, la réalisation a révélé leur rôle plus complexe : un groupe doit mémoriser les filtres de sélection, maintenir sa cohérence lors des modifications et clarifier ses membres. Des incohérences sont apparues (membres en base non affichés ou inversement) dues à des filtres actifs, nécessitant une clarification de la conception.

2. Objectifs de conception

Nous avons établi la colonne *groupe_id* de la table *CANDIDAT* comme unique source de vérité pour l'appartenance à un groupe.

Les filtres utilisés pour créer un groupe sont conservés dans des entités *Critere*, stockant le libellé et la valeur/intervalle, permettant de reconstruire les filtres à l'édition et de documenter la construction du groupe.

L'architecture contrôleur – service – repository a été respectée. Les contrôleurs gèrent les requêtes *HTTP* et la validation de base. Le *GroupeService* contient la logique métier (création, mise à jour, suppression). Les repositories gèrent l'accès *SQL* aux tables *GROUPE*, *CRITERE*, *FILTRE* et *CANDIDAT*, assurant lisibilité et évolutivité.

B. Vue d'ensemble de la chaîne de traitement

1. Scénario général

La création d'un groupe, similaire aux filtres de dossiers, utilise les cases à cocher de la liste des candidats affichée par *tableau.js*. Le script *groupe.js* récupère les codes candidats sélectionnés pour ouvrir une modale de création (nom, couleur, note). La validation envoie une requête *fetch JSON* à *creerGroupe.php*, contenant le nom, la couleur, la note, la liste des candidats et les filtres actuellement appliqués (*filtresCourant* ou *serialiserFiltres()*).

2. Rôle des contrôleurs

Les fichiers *creerGroupe.php* et *modifierGroupe.php* dans le dossier publicinstancient les contrôleurs *CreerGroupeController* et *ModifierGroupeController*. Chaque contrôleur vérifie que la requête arrive en *POST* et que l'utilisateur connecté est administrateur. Il lit ensuite le corps *JSON*, extrait les champs attendus et

applique une première validation : nom non vide pour la création, identifiant de groupe strictement positif pour la modification, présence d'au moins un code candidat, note optionnelle mais bornée à [0 ; 20], etc.

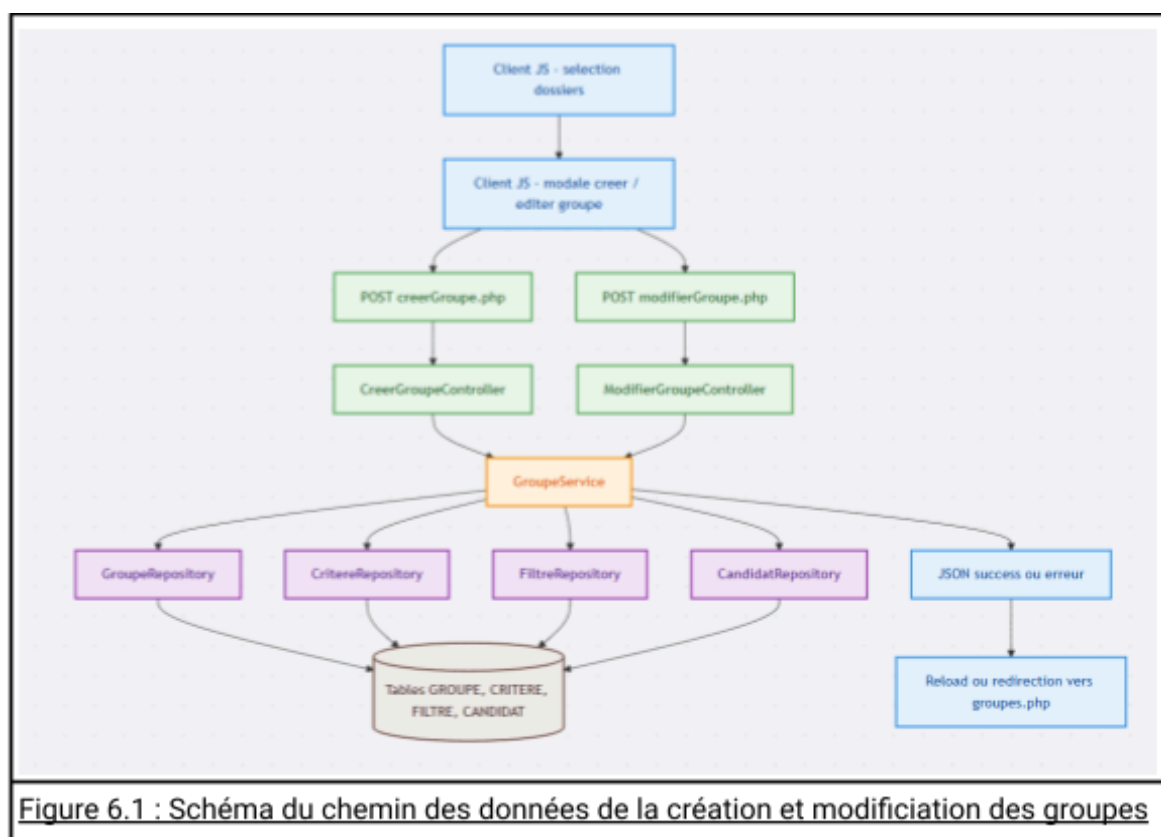
Une fois ces vérifications faites, le contrôleur délègue la suite au service `GroupeService`. Il entoure cet appel d'un bloc `try/catch` pour capturer les erreurs éventuelles. En sortie, il renvoie toujours un JSON simple du type `success / message`, que le JavaScript affiche ou exploite.

3. Rôle du service et des repositories

Le service *GroupeService* gère les groupes. Il utilise *GroupeRepository*, *CritereRepository*, *FiltreRepository* et *CandidatRepository* pour manipuler les données.

Pour la création, *creerGroupeDepuisSelection* ouvre une transaction, crée et persiste le *Groupe* (nom, couleur, note), enregistre les critères et les liens *FILTRE*, puis affecte le groupe aux candidats sélectionnés. La transaction est validée ou annulée en cas d'erreur.

Pour la modification, *mettreAJourGroupe* charge le groupe, met à jour ses propriétés, supprime les anciens critères/liens, recrée les nouveaux, et synchronise l'affectation aux candidats (ajout/retrait).



C. Architecture des fichiers liés aux groupes

1. Organisation back-end

L'architecture des fichiers de gestion de groupes est la suivante :

- controllers : DossiersController (page des dossiers/filtres candidats), GroupeGetController (liste des groupes paginée), CreerGroupeController et ModifierGroupeController (endpoints JSON).
- services : GroupeService (logique métier *CRUD*, conversion filtres/critères), GroupeFiltreService et DossierFiltreService (description des filtres).
- repositories : Manipulent les tables : GroupeRepository (*GROUPE*), CritereRepository (*CRITERE*), FiltreRepository (liens groupes/critères) et CandidatRepository (*groupe_id* de *CANDIDAT*).
- entities : Classes *Groupe* (*nom*, *couleur*, *note*, *critères*, *membres*), *Critere* (*libellé*, *valeur*, *bornes*) et *Candidat* (avec *groupe_id* éventuel).

2. Vues et scripts côté client

Les vues Twig (dossiers.html.twig, groupes.html.twig) et les composants partagés (tableau.html.twig, filter_open.html.twig) sont dans views et assets. Les points

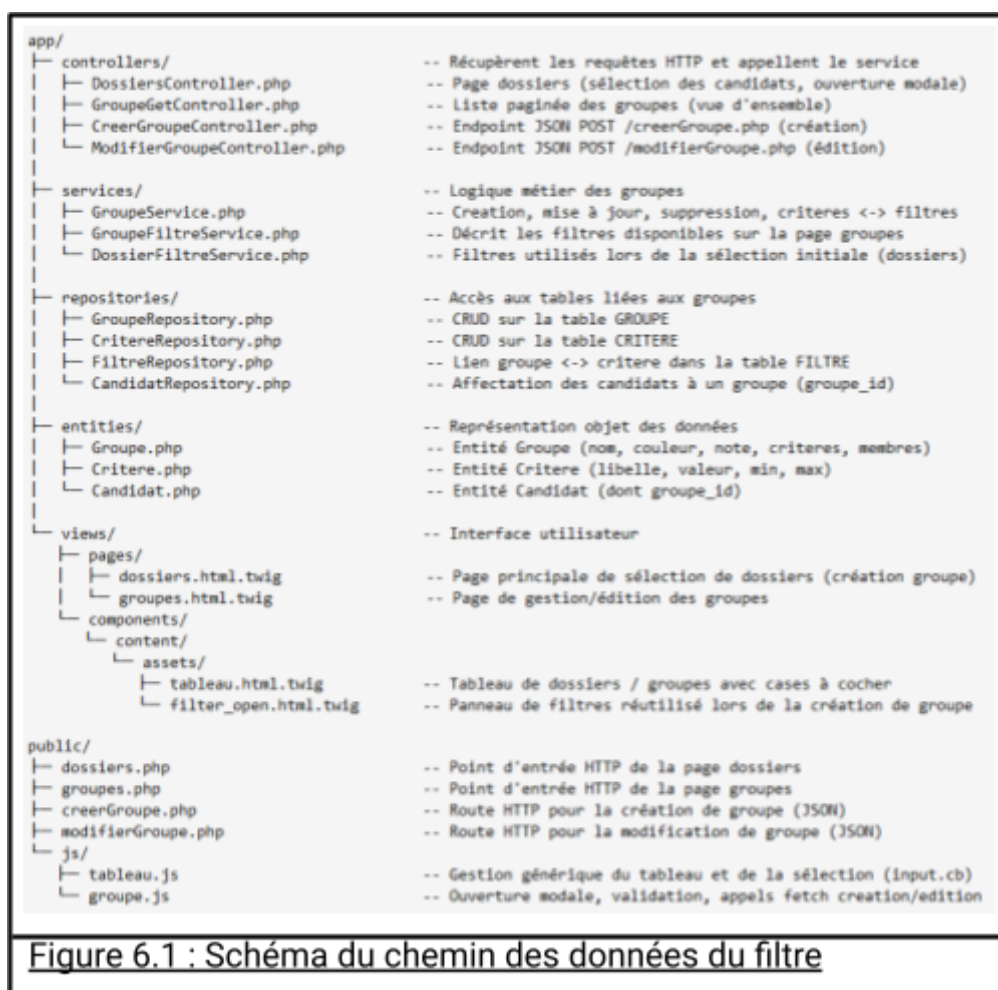


Figure 6.1 : Schéma du chemin des données du filtre

d'entrée PHP ([dossiers.php](#), [groupes.php](#)) et les routes JSON ([creerGroupe.php](#), [modifierGroupe.php](#)) se trouvent dans public. Le dossier js contient les scripts clients pour les tableaux ([tableau.js](#)) et la gestion des groupes ([groupe.js](#)).

D. Détails de la création et de la modification

1. Création de groupe côté client

La création commence toujours par une sélection de dossiers sur la page des dossiers. L'utilisateur coche les lignes qui l'intéressent et clique sur « Créer un groupe ». Le script `groupe.js` lit les cases cochées avec ``getSelectedCodes()``, affiche le nombre de dossiers sélectionnés dans la modale et ouvre cette modale.

Lors de la soumission du formulaire, le script valide les champs (nom obligatoire, note optionnelle bornée, au moins un dossier sélectionné), récupère les filtres courants, construit un JSON et l'envoie en POST vers ``creerGroupe.php``. En cas de succès, il recharge la page, ce qui remet à jour le tableau et les informations associées.

2. Modification et synchronisation des membres

La modification se fait depuis la page des groupes. L'utilisateur choisit un groupe, accède à un formulaire pré-rempli, puis adapte le nom, la couleur, la note, les membres et les filtres. Le script [groupe.js](#) applique les mêmes règles de validation que pour la création, récupère les filtres courants et envoie un JSON vers ``modifierGroupe.php``.

Le service met ensuite à jour le groupe, reconstruit ses critères et synchronise ses membres avec la nouvelle sélection. Au final, la table ``CANDIDAT`` reste toujours cohérente avec ce que l'utilisateur voit à l'écran, et la liste des critères stockés pour le groupe reflète fidèlement les filtres appliqués au moment de la sauvegarde.

VII. La carte, du back au front

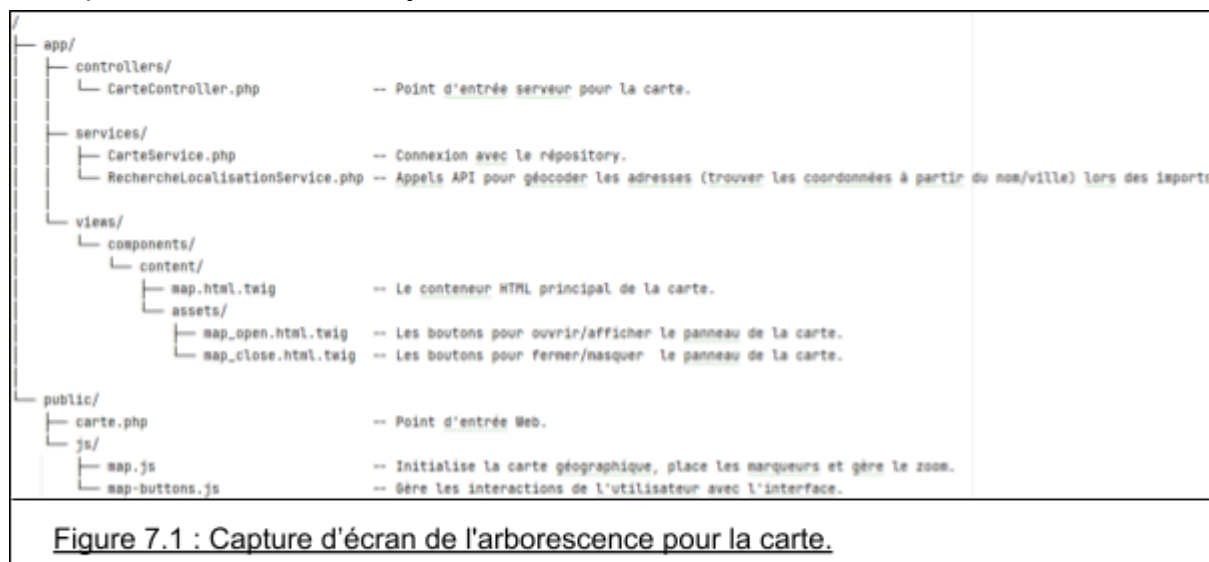
Nous allons expliquer les api choisies, la méthode de chargement et de récupération des données.

A. Fichiers utilisée

On a voulu diviser la map en divers fichiers.

Par exemple 2 repositories, 1 pour appeler les api et mettre à jour les position géographique et distances des établissements et un second qui n'est relié que pour appeler les données afin de les renvoyer au fichier map.js via son fetch avec le verbe HTTP GET.

Ou encore, pour les .js, nous avons 1 qui gère les boutons afficher/masquer et un pour la création, mis à jour de la carte.



B. Choix technique

1. Les API utilisées :

Pour la gestion des API, nous avons privilégié l'[API du gouvernement français](#). Cette est utile pour localiser des adresses françaises en utilisant la commune, le département, le code postal ou une adresse complète, ce qui correspond aux informations de nos établissements. Un avantage majeur est la gratuité et l'efficacité de cette API, pour envoyer des fichiers CSV (générés avec PhpSpreadsheet) et obtenir un retour un fichier avec les données de localisation complétées.

Pour les cas où l'API française ne trouve pas la localisation, nous utilisons [Photon](#). Ce service est plus précis et de portée mondiale, ce qui est très utile pour localiser des établissements à l'étranger.

Photon n'était pas notre premier choix car nous avons d'abord considéré Nominatim. Cependant, [Nominatim](#) s'est avéré plus complexe à utiliser, et un test

avec une adresse située à Hong Kong n'a pas été concluant. Nous avons opté pour Photon, avec les requêtes de localisation une par une, et un délai entre chaque demande pour éviter d'être banni.

2. La carte choisi :

Pour la carte nous avons opté comme précisé dans le sujet [LeafletJs](#) avec laquelle nous avons personnalisé la map et aussi créer des marker/layers pour gérer le zoom et dezoom.

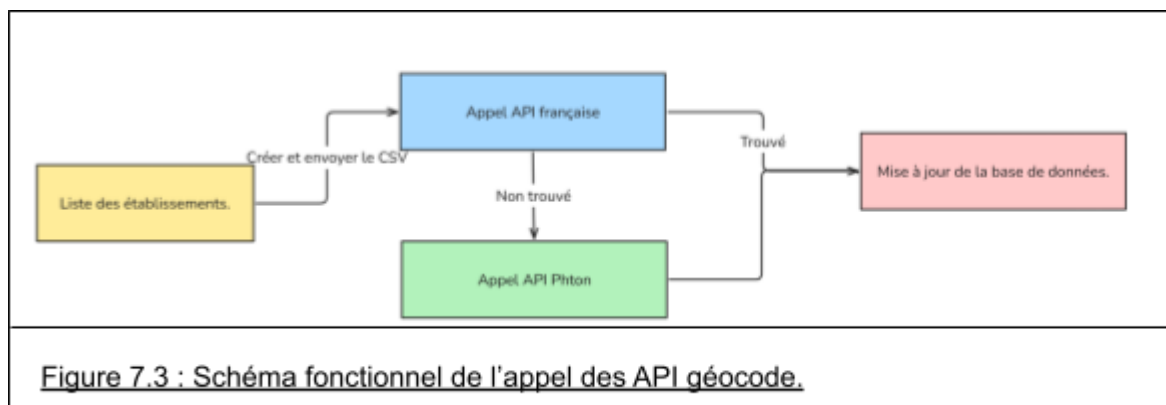
C. utilisation d'api

Pour appeler les api nous faisons cela uniquement dans RechercheLocalisationService.php, cette classe est créée par ImportService.php qui va appeler la méthode de création du fichier, ajouter les établissements s'ils n'existent pas après les comparaisons et enfin appeler la méthode pour lancer la recherche et ensuite parcourir le tableau. Pour chaque localisation non trouvée on va appeler photon derrière pour tenter de définir la localisation.

L'appel aux API est centralisé dans la classe RechercheLocalisationService.php. Cette classe est instanciée par ImportService.php, qui appelle les méthodes pour la création du fichier, ajout des établissements après comparaison si non existants (`$this->serviceRechercheLocalisation->addEtablissement($etablissement)`), lancement de la recherche (`$retour = $this->serviceRechercheLocalisation->appelerApi($fichier);`), puis parcours du tableau de résultats (`$this->serviceRechercheLocalisation->parcoursTableau($retour)`).

Pour toute localisation non trouvée initialement, un appel est effectué à *Photon* pour tenter de la déterminer.





D. La création de la map

Le front de la map (balisage) est fait exclusivement dans [map.html.twig](#) et pour les (link des librairies) dans [dossier.html.twig](#). Pour son contenu et sa gestion, tout est dans [map.js](#).

L'initialisation de la map se fait au démarrage, dès l'arrivée sur la page [dossier.php](#). A son lancement, on appelle *initMap()* et *getCarte()* pour créer la map, son style et ces couches puis *getCarte()* pour appeler notre api qui nous retourne les données des établissements.

Pour le fetch de notre api nous appelons notre api ([carte.php](#)) pour qu'elle appelle son controller qui vérifie nos permissions et données saisies puis si elles sont correctes, on renvoie les établissements correspondant à ces critères.

```
no usages & HERMILLY Joshua
public function jsonSerialize(): array
{
    return
    [
        'etablissement_id'      => $this->etablissement_id,
        'etablissement_nom'     => $this->etablissement_nom,
        'etablissement_pays'    => $this->etablissement_pays,
        'etablissement_code_postal' => $this->etablissement_code_postal,
        'etablissement_commune' => $this->etablissement_commune,
        'etablissement_departement' => $this->etablissement_departement,
        'etablissement_latitude' => $this->etablissement_latitude,
        'etablissement_longitude' => $this->etablissement_longitude,
        'etablissement_distance' => $this->etablissement_distance,
        'nb_candidats'          => $this->nb
    ];
}
```

Figure 7.5 : données renvoyé par l'api en format JSON.

Pour chaque établissement donné on va appeler la méthode pour ajouter un marker correspondant à l'établissement donné.

```
/*-----*/  
/* AJOUTER MARQUEUR */  
/*-----*/  
Show usages 3 HERMILLY Joshua  
function ajouterMarqueur(etablissement) : void  
{  
    const lat = etablissement.etablissement_latitude;  
    const lon = etablissement.etablissement_longitude;  
  
    // Coordonnées ?  
    if (lat === null || lon === null || (lat === 0 && lon === 0))  
    {  
        //console.warn('Pas de coordonnées pour: ${etablissement.etablissement_nom}');  
        return;  
    }  
  
    const marker = L.marker([lat, lon]);  
    const nomEtab = etablissement.etablissement_nom ;  
    const codePostal = etablissement.etablissement_code_postal;  
    const commune = etablissement.etablissement_commune ;  
    const nb_candidat = etablissement.nb_candidats ;  
  
    mapCluster.addLayer(marker);  
    marker.bindPopup  
    (`  
        <p><strong>${nomEtab}</strong></p>  
        <p>${codePostal}</p> ${commune}</p>  
        <p><strong>Nombre d'étudiants : </strong>${nb_candidat}</p>  
    `);  
}
```

Figure 7.6 : Ajouter un marqueur sur la map.

VIII. Apprentissage retenue

Ce projet nous a beaucoup appris, notamment sur l'importance d'une bonne conception.

Une conception solide est essentielle pour éviter de devoir revenir sur les fondations, en particulier la base de données. Nous avons passé beaucoup de temps à effectuer plusieurs corrections sur cette dernière, ce qui a représenté une perte de temps considérable, plusieurs jours.

Cependant, le *design*, qui était déjà défini, a permis de mieux structurer les choses dès le départ et d'éviter d'avoir à les remodifier en cours de route.

Ensuite, une réalisation a toujours des oublies ou des imprécisions sur la phase de conception. C'est dans ces moments qu'il faut déterminer en groupe les choix et chemins à suivre de façon logique et propose pour ne pas avoir à revenir en arrière dont notamment faire des teste comme avec l'api des cartes (Photon et Nominatim) qui avec leurs maquette a permis de corriger cela en pas longtemps.

De plus, ce projet a été l'occasion d'acquérir une maîtrise de l'utilisation des API externes, ainsi que de la conception d'API internes (telles que celle de gouvernement, Photon, dossierGet, etc.). Il a également nécessité l'utilisation de librairies spécifiques, notamment PhpSpreadSheet.

Au-delà des compétences techniques, ce projet a renforcé nos *soft skills*, notamment en collaboration et communication. L'importance de réunions régulières et de la mise en commun des avancées. Nous avons appris à gérer et prévenir les désaccords qui surviennent en équipe, en privilégiant une écoute et la recherche de solutions logiques. Un autre apprentissage fut la nécessité d'un feedback continu et la recherche d'information, que ce soit en demandant conseil aux professeurs, en se renseignant sur la documentation, ou en échangeant avec les autres groupes.

IX. Conclusion

Ce projet, réalisé dans le cadre de la SAE 4.01, nous a permis de concevoir et de développer une application web visant à optimiser le temps de profilage des dossiers Parcoursup pour le département Informatique.

Notre équipe a mené à bien la réalisation de cette SAE, livrant une application dotée de toutes les fonctionnalités essentielles attendues et s'appuyant sur l'architecture logicielle fournis (Contrôleur-Service-Repository). Nous avons structuré notre travail d'équipe en utilisant la méthodologie Git Flow. Le développement d'un système de filtres modulaires a permis de centraliser la logique métier. L'intégration d'un tableau modulaire et d'une carte (via Leaflet) assure une lecture claire et une visualisation simple des candidats.

Ce projet a soulevé des défis techniques, notamment des modifications de la base de données rendues nécessaires par des erreurs initiales concernant la structure des formations d'élèves et le rattachement des établissements à leurs localisations.

Afin de corriger les échecs de localisation pour certaines adresses, nous avons dû combiner les services de plusieurs API (Gouvernement français et Photon). Et pour la gestion d'import et export nous avons intégré la librairie PhpSpreadsheet pour garantir cela. Ces technologies nous ont permis d'en apprendre plus sur des possibles utilisations dans le monde du travail.

Pour finir, cette SAE a été extrêmement enrichissante, pour consolider nos compétences en développement full-stack, en conception de bases de données et en collaboration. L'application finale représente une solution concrète qui répond aux exigences initiales.